

PsyFEM

Extended Public Technical Overview

Curated disclosure for technical evaluation

Author: Étienne Prieckaerts

Publication date: June 2026

Repository scope: public concept and bounded implementation status

This document intentionally omits source code, exact algorithms, internal schemas, parameter values and reproduction-critical execution details.

Abstract

PsyFEM is a proprietary, deterministic and persistent structure and memory engine for processing episodic or event-like information over longer periods of time. The system maintains an evolving working state, updates relationships incrementally and forms higher-order representations through multiple stages of structural condensation.

The first public PsyFEM materials, published in 2025, described the conceptual foundation. Since then, the work has progressed into a functional technology prototype. This document explains the prototype at a level intended to be useful to technical readers while intentionally omitting the implementation details required to reproduce the system.

1. What PsyFEM Is

PsyFEM is not a language model and not a wrapper around a language model. It is a separate persistent learning and structure engine. Language models or other external systems can be connected around it, but they are not the internal learning core.

The engine processes observations as episodes or events. Over time, these inputs contribute to a changing graph-like internal state. Relationships can strengthen, weaken or be reorganized. Higher-order structures can be formed from lower-level observations. Read-oriented analysis paths expose selected learned structures without turning every query into a new learning event.

The internal state space can represent emotional, semantic or other domain-specific dimensions. The emotional interpretation was central to the original concept, but the prototype is designed as a more general structure and memory engine.

2. Public Architecture Model

At a public architectural level, PsyFEM can be understood as eight cooperating areas:

- 1 Episode and event intake: observations enter the system as time-related units.
- 2 Persistent dynamic graph: units and derived structures are connected through adaptive relationships retained across sessions.
- 3 Incremental learning loop: incoming information updates affected structures instead of rebuilding a blank state for every request.
- 4 Multi-stage condensation: lower-level episodes can contribute to progressively more abstract structural representations.
- 5 Operational control: long-running processing, resume behavior and runtime state are managed separately from the graph representation itself.
- 6 Read-oriented analysis: learned structures can be inspected through dedicated exploration, reporting and audit surfaces.
- 7 Guarded adaptation: selected operational settings can be evaluated and applied through controlled profile paths with review controls and history records.
- 8 Integration demonstrator: an external live event stream can be connected to operational learning paths and inspected through a dedicated control surface.

This description is intentionally architectural. The private implementation contains the exact update equations, gating logic, data schemas, configuration values and execution sequences.

3. Learning Mechanism Families

The prototype combines known mechanism families in a specific persistent architecture. The following descriptions identify how the system learns without publishing a reproduction recipe.

3.1 Episodic accumulation

Incoming observations are retained as episodic units rather than reduced immediately to a single answer or transient prompt context. This gives later processing a time-related evidence base and allows the structure to evolve across sessions.

3.2 Adaptive graph relationships

The internal working state is graph-based. Relationships between units are not treated as permanently fixed links. Their effective strengths can change as the system processes new input and revisits existing structure.

3.3 Activation and propagation

Input can activate a local part of the internal structure. Influence can then propagate through connected regions. This allows related material to affect processing without requiring every stored unit to participate equally in every step.

3.4 Co-activation reinforcement

The learning loop contains Hebbian-style reinforcement: structures that repeatedly participate together can gain stronger association. The public statement is limited to the mechanism family. Exact update rules, coefficients and guards remain private.

3.5 Decay, forgetting and pruning

PsyFEM does not treat all stored material as equally permanent. The prototype includes mechanisms for weakening or limiting structures over time and for controlling unbounded growth. This is relevant for long-running memory behavior and operational cost.

3.6 Recoding and reconsolidation

Existing representations are not necessarily immutable. Later context can influence how earlier material is weighted or interpreted. This supports controlled reconsolidation rather than append-only storage alone.

3.7 Multi-stage abstraction

The current engine contains a layered condensation path. Publicly, this can be described as:

- L0: episodic or event-level input;
- L1: local structural condensation;
- L2: category- or cluster-level organization;
- L3: broader contextual structure;
- L4: meta-level structure and control-oriented signals.

The levels describe responsibilities, not a complete algorithm. The private implementation defines the conditions, budgets, fallbacks and interactions used to build and update them.

3.8 Feedback and regulation

Higher-level context can influence lower-level processing through controlled feedback paths. This is not equivalent to unconstrained self-modification. The implementation contains modes, guards and diagnostic behavior that separate observation from active intervention. Selected adaptation paths are intentionally controlled and auditable.

3.9 Graph-aware retrieval

Read-oriented memory access is not limited to plain keyword matching. The prototype uses local candidate selection and graph-aware ranking behavior so that learned structure can influence retrieval. Exact scoring composition and optional diffusion details remain private.

3.10 Auditability and replay

The engine is designed for inspectability. Processing runs can expose progress, status, issue and snapshot artifacts. Replay and resume paths make it possible to analyze learning behavior over time instead of treating the internal state as an opaque black box.

4. What Is Architecturally Different

PsyFEM uses familiar building blocks, but it combines them differently from several common market patterns. This section describes architectural distinctions, not a claim that competing systems are inferior or that no other research system explores related ideas. It is not an exhaustive landscape study or a legal novelty opinion.

4.1 Compared with vector retrieval and RAG

Vector retrieval systems typically index chunks or records and return semantically similar results for a query. Official OpenAI Retrieval documentation describes vector stores as indices for semantic search. Pinecone documentation similarly describes semantic search as retrieving records that are close in dense-vector space.

PsyFEM can support retrieval-like use cases, but its core responsibility is broader: the persistent internal structure itself changes through episodic processing, reinforcement, decay, reconsolidation and layered condensation. Retrieval is one

read-oriented surface over an evolving learned state, not the complete learning model.

4.2 Compared with graph databases and knowledge graphs

Property graph systems represent entities as nodes and relationships, often with labels and properties. This is a powerful way to store and query structured relationships.

PsyFEM also uses graph structure, but it is not merely a graph database frontend. The engine contains a learning loop in which time-related episodes, adaptive relationships, activation, condensation and regulation jointly affect the persistent state. The distinction is dynamic learning behavior rather than graph storage alone.

4.3 Compared with agent-memory frameworks

Agent frameworks commonly persist thread state, checkpoints or long-term memory documents. LangGraph documentation, for example, distinguishes thread-scoped short-term memory from long-term stores organized by namespace and key.

PsyFEM is not primarily a checkpoint or profile store. It is designed to derive and update internal structure across observations. It can sit behind an agent interface, but the graph dynamics and multi-stage learning remain a separate subsystem.

4.4 Compared with LLM-only personalization

Prompt history, saved facts and retrieved documents can improve continuity, but they do not automatically create a deterministic structural learning engine. PsyFEM keeps its own persistent state and can be inspected independently of any connected language model.

5. Implemented Prototype Surfaces

The current private implementation includes multiple surrounding surfaces. The counts below describe the reviewed workspace as of 2 June 2026.

5.1 Core and orchestration

- Core package: 88 Python files and approximately 36,100 physical Python lines.
- Orchestration area: 33 Python files and approximately 48,900 physical Python lines.
- Tool and analysis area: 117 Python files and approximately 29,400 physical Python lines.
- Measured implementation and tool scopes combined: approximately 114,400 physical Python lines.

These counts exclude tests and deliberately exclude historical snapshot and refactor trees. Physical line counts are inventory indicators, not quality scores.

5.2 API surfaces

Two FastAPI-based surfaces are implemented:

- Knowledge-oriented API: 25 declared HTTP routes. It is read-mostly and covers knowledge queries, simulation-oriented queries, hypotheses, structural spaces, workspace selection and selected operational actions.
- Agent bridge API: 33 declared HTTP routes. It covers observation, ingestion, query and simulation modes, learning-run control, workspace access and selected integration paths.

Together, the reviewed API files contain 58 declared HTTP routes. Not every route has the same maturity or the same direct test depth.

5.3 User interfaces

Four Streamlit-based UI entry surfaces are present:

- an operational dashboard;
- a local UI and service hub;
- a read-oriented knowledge explorer and observatory surface;
- a dedicated integration control surface.

The UI layer is broader than its direct UI automation depth. Backend contracts and helper paths are more strongly validated than every visual workflow.

5.4 CLI and tool paths

Reviewed command-line and tool families include:

- replay and continued-learning runs;
- ingestion from multiple input forms;
- read-oriented knowledge exploration;
- hypothesis generation and persistence;
- evaluation and report generation;
- archive comparison and A/B-style analysis;
- controlled profile evaluation and adaptation support.

At least eight central tool paths were individually inspected during the internal audit. The private workspace contains additional helpers and operational scripts.

6. Internal Validation Inventory

The private workspace uses `pytest` as its primary test runner. The default test configuration points to the active tests/tree and keeps historical snapshot trees separate from current coverage statements.

As inventoried on 2 June 2026:

- primary active suite: 241 test files;
- primary active suite: 1,107 discovered test functions;
- separate supplemental QA tree: 6 test files with 17 discovered test functions;
- dedicated E2E directory: 11 test files;
- dedicated integration directory: 21 test files with 44 discovered test functions;
- dedicated validation directory: 12 test files with 40 discovered test functions;
- dedicated forensics directory: 7 test files with 9 discovered test functions;
- dedicated tool-test directory: 28 test files with 126 discovered test functions;
- E2E- or system-smoke-oriented filenames across the active suite: 14.

The inventory includes tests for areas such as:

- workspace lifecycle and persistence;
- read-only versus mutating query behavior;
- simulation isolation;
- layered structure formation;
- replay visibility and silent-failure detection;
- crash recovery and resume behavior;
- runtime configuration changes;
- process-mode workers;
- API bridge behavior;
- audit reporting and analysis helpers;
- CLI and tool paths.

A focused internal closure run executed six newly added direct-test files for previously open target paths. Result: 16 of 16 tests passed.

6.1 Selected dated evidence

The current publication uses a recency hierarchy. The central code-based audit series from 31 May and 1 June 2026 is treated as the primary evidence source. Older internal validation material is used only as scoped supporting evidence.

Selected recorded internal runs include:

- a focused silent-failure and core-path validation recorded on 21 May 2026 with 12 of 12 and 60 of 60 scoped tests passing;
- the central audit closure run recorded on 1 June 2026 with 16 of 16 direct tests passing for previously open target paths;
- live-integration hardening and additional regression tests added on 1 June 2026.

A broader validation harness recorded on 11 May 2026 remains internally documented but is not presented here as a single aggregate success figure. Its scope combines different evidence classes, including synthetic or temporary-workspace-constrained material.

These figures are intentionally precise about scope. They do not claim that all 1,107 test functions were executed together in a fresh public release run, that every UI workflow has E2E coverage or that the complete system is production-certified.

7. Development Status

The appropriate current classification is:

Functional technology prototype under active development.

The internal audit supports the statement that a real implementation exists and that central core, orchestration, persistence, analysis and validation surfaces are present. Documented residual risks remain, especially around the breadth of integration validation and direct UI workflow testing.

The public materials do not claim:

- production certification;
- complete validation of all integrations;
- complete test coverage;
- independently reproduced benchmark superiority;
- suitability for safety-critical deployment.

8. Potential Application Contexts

Depending on the surrounding integration and domain model, PsyFEM may be relevant to:

- persistent memory for adaptive software systems;
- long-horizon context and experience analysis;
- inspectable agent-support systems;
- structured observation and event analysis;
- research into dynamic state, memory and adaptive behavior.

These are application contexts, not claims of completed products.

9. Disclosure Boundary

This extended public overview intentionally explains mechanism families and measured engineering surfaces while omitting the material required for direct reproduction. Not published are:

- source code and executable examples;
- formulas and exact update algorithms;
- internal data schemas and artifact formats;
- parameter values, thresholds and optimization recipes;
- complete internal module maps and call chains;
- private datasets, logs and runtime artifacts;
- complete endpoint catalogs and private operational instructions;

- complete test matrices, internal risk maps and private audit records;
- benchmark setups and detailed measurement series.

Additional technical material may be shared selectively in an appropriate confidential setting.

10. References for Market-Pattern Comparison

The market-pattern comparison was checked against public official documentation on 2 June 2026:

- 1 OpenAI Retrieval documentation: <https://platform.openai.com/docs/guides/retrieval>
- 2 Pinecone semantic search documentation: <https://docs.pinecone.io/guides/search/semantic-search>
- 3 Neo4j graph database concepts: <https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/>
- 4 LangGraph memory overview: <https://docs.langchain.com/oss/python/langgraph/memory>

11. Conclusion

PsyFEM has progressed beyond its initial concept stage. As of June 2026, it exists as an internally reviewed technology prototype with persistent state handling, incremental learning, adaptive graph relationships, multi-stage structural condensation, replay-oriented auditability and separate analysis surfaces.

This document provides a technically meaningful public explanation of the prototype while preserving the implementation details that distinguish the proprietary system.